Number: P3818R1
Date: 2025-09-03
Audience: Library Evolution
Target: C++26
Author: Hana Dusíková

P3818R1: constexpr exception fix for potentially constant initializ

This paper proposes fix to surprising silent code breakage introduced by P3068 *"constexpr exceptions"* interacting with potentially-constant initialization [expr.const]. This was found by Lénárd Szolnoki and discussed at library and core wording groups reflectors.

Problem described in this paper is a significant silent breakage with simple fix, I'm asking chairs of LEWG and LWG to treat this with high-priority, as there will probably be some NB comments asking WG21 to fix the described issue.

Revision history

- R0 → R1: added section discussing changes proposed in P3820, added comparison table to explaing subtle differences between various types of variables and their initialization, added comparison table between P3818 and P3820 proposals.

The problem to fix

C++ has many corner cases and this one is one of them. A constant variable (marked `const`) which is integral or enumeration type is upgraded to `constexpr` variable silently if its initialization succeed in constant evaluation. This is usually unobservable, because you can't reference anything around to succeed.

```
auto function_returning_empty_array() {
  const int n = calculate_size_needed(); // this needs to be a constant evaluat
ed => constexpr
  return std::array<int, n>{}; // so we can change type based on `n`
}
```

This is a problem for `constexpr` exceptions, which needs `constexpr` marked functions in order for them work inside constant evaluation. But once marked `constexpr` a function can be evaluated there, which is a problem for two functions added by[P3068](#) (`std::uncaught_exceptions` and `std::current_exceptions`) as these don't have dependency on any local variable which would disallow constant evaluation. These have only an implicit dependency on local context which allows them to be succesfully evaluated in `const` variable potentially-constant initialization.

This potentially-constant initialization starts as any constant evaluation a new context, in which there are no unrolling or current exception, so these function will return constant which is something else user wants.

```
try {
  // some exception throwing code
} catch (const std::exception & exc) {
  const bool has_exception = (std::current_exception() != nullptr); // no curre
nt exception in a catch handler?!
  static_assert(has_exception == false); // success here
}
```

Variable `has_exception` is silently upgraded to a `constexpr` variable. And records different context than a user expects. Another even more scary example:

```
struct transaction {
  // ...
  void cancel() { /* revert changes */ }

  ~transaction() {
    const bool unrolling = std::uncaught_exceptions() > 0;

    if (unrolling) { // this will never be evaluated
      log("exception was thrown in a transaction => cancel()");
      cancel();
    }
  }
}
```

Note: I know this example should contain call in `std::uncaught_exceptions()` so we can actually know if an there is a new exception, but in order to simplify it I did what I did.

In previous example `std::uncaught_exceptions() > 0` is constant evaluated in a vacuum, even sooner than the destructor `transaction::~transaction()` is finished parsing. And because in that specific constant evaluation, there is no uncaught exception, it will return 0, *obviously*. The whole `unrolling` becomes `constexpr`, and your transactions will **never** cancels. This is really scary.

Explainer table

	potentially-constant initialization (sometimes constant evaluated without visible clue)			
	C++23	C++26 (status quo)	P3818 (this paper)	P382((Lénárd's p
`const int n = uncaught_exception()` (local variable)	number of exceptions (during runtime evaluation)	always `0` (constant evaluated during parsing)	number of exceptions (during runtime **and constant** evaluation)	number of ex (during **runtime evaluat** compile in constant e
`static const int n = uncaught_exception()` (local static variable)	number of exceptions (during **first** runtime evaluation)	always `0` (constant evaluated during parsing)	number of exceptic (during **first** runtime evaluation, `static` not available in constant evaluate	
`const int n = uncaught_exception()` (object member variable)	number of exceptions (during evaluation of object's constructor, never w evaluation, unchanged)			
	explicit constant evaluation			
	C++23	C++26 (status quo)	P3818 (this paper)	P382((Lénárd's p

`constexpr int n = uncaught_exception()`
(local constant)
	N/A	always `0`
		(constant evaluated during parsing)
				failure to compiler
				`constexp`
				`uncaught_exceptio`

`static constexpr int n = uncaught_exception()` (static constant)	N/A	always `0` (constant evaluated during parsing)		use ne `consteval_uncaugh` functic
`static_assert(uncaught_exception() == 0)`	N/A	always `true` (constant evaluated during parsing)		

`auto obj = some_template<uncaught_exception()> {}`	N/A	`some_template<0>` (constant evaluated during parsing)	

The code in table is intentionally visible, I don't expect users to write `constexpr int n = std::uncaught_exceptions()` but I do expect users to write code which is using the function deep inside the code, not visibly.

`constexpr` variables are not a problem

It can be surprising to some users a local `constexpr` variables are not observing local evaluated context. But it's long established all `constexpr` variables are starting completely new evaluation without any evaluation context at site of declaration (it can use template variables, other `constexpr` variables, but not any local variable).

We need to keep `constexpr` marked functions in order to have the constexpr exception functionality fully working during constant evaluation (storing exceptions temporarily). Failing to do so would make a somehow arbitrary functionality of language again impossible to use and for users to go around, which I strongly prefer to avoid so.

In order to do we must make the potentially-constant initialization evaluation fail when it reaches these two function in question. It's a small surgical and mostly inobservable change which saves us from the silent code change when upgrading to 26. But also it's much better than just removing `constexpr`.

Proposed solution

Keep `constexpr` on both methods (`std::uncaught_exceptions()` and `std::current_exceptions`) and **disallow them** to be constant evaluated explicitly only **in** potential-constant initialization [expr.const].

This is a minimal and implemented solution which doesn't limit functionality, but removes the break.

Possible alternatives

Much larger but probably breaking solution

We could deprecate and later remove `potentially-constant` from language. This would make C++ much less surprising, but it will be probably a significant breaking change, altrough not really hard to fix (just make your `const` variables which suddenly failed to compile `constexpr` and you are good to go.)

Because of the large impact, this is not proposed.

Alternative and somehow arbitrary solution

Alternative solution would be to disallow these two functions not just in potentially-constant initialization, but in any initialization. But that would make the functionalily severely limited and arbitrary for users and they would need to go around it, which would lead to more complicated code as `constexpr` variables are most common form of current meta-programming where it's used to precalculated values and tables.

Duplicate functions under different name specially for constant evaluation

This is what P3820R0 proposes. Removing `constexpr` from `uncaught_exceptions` (not from `currrent_exception` as that one is not touched by the paper) and introduce a same function under similar name. I think that's a bad solution, because these functions are doing same thing, problem is interaction with language rules. Also I really don't think we should introduce same functionality which then would need to be `if consteval`-ed on every place where we want to use code for both runtime and constant evaluation. It will lead to code like this:

```
constexpr foo::~foo() {
  if consteval {
    /* const */ int num_of_exceptions = std::consteval_uncaught_exceptions();
    if (num_of_exceptions != num_of_exceptions_before) {
      // do something now we know there is an exception unrolling
    }
  } else {
    const int num_of_exceptions = std::uncaught_exceptions();
    if (num_of_exceptions != num_of_exceptions_before) {
      // do something now we know there is an exception unrolling
    }
  }
}
```

Previous example shows multiple problems, even if we do introduce new function, we can still can't use it in `const int` variable initialization, because it would create new constant evaluation and fold into constant. We must avoid `const int` there completely and we must teach it then.

I fully expect users to write something like following function:

```
constexpr int my_uncaught_exceptions() {
  if consteval {
    // for some reason committee gave us this function,
    // but I need this to initialize variable, so I can't use
    // if consteval there
    return std::consteval_uncaught_exceptions();
  } else {
    return std::uncaught_exceptions();
  }
}
```

And later someone will use this function in `const int n = my_uncaught_exceptions` initialization and gets burned anyway. Therefore I argue against creating new variable, instead I propose taking proposal of this paper, which will make sure there is no breaking change.

Runtime and constexpr exception don't have different semantic

There is no difference semantic in how runtime and constexpr exceptions behave. Hence providing different namely function to do the same just in different context will lead to confusion and previously describe problems. Difference is on the language level between local variables (`int n = ...`) and local constants (`constexpr int n = ...`) and when these are initialized and evaluated. With future support for compile time debug printing from P2758 users will be able observe when is what code evaluated, and will understand that code evaluated in their compiler can't logically know about number of uncaught exceptions during runtime evaluation.

One function name is needed

If we split functions to two, we are forcing users to write their own "one common function". If they want to continue support both types of their exceptions in pattern where number of unrolling exceptions is stored in a member variable for later comparison in destructor. This shows it's same functionality and it would burden users.

Positive / negative comparison

Following table compare positives and negatives of this paper and P3820.

C++26's status quo	
positives	**negatives**
• constexpr and runtime exceptions utility functions works same in both environments	• silent breakage of code for "always use `const`" people `const int n = std::uncaught_exceptions()` is constant evaluated during parsing

	• `constexpr int n = std::uncaught_exceptions()` can be confusing for some

P3818: "constant when: not within potentially-constant initialization"

positives	negatives
• no breakage in existing code • constexpr and runtime exceptions utility functions works **mostly** same in both environments • `const int n = std::uncaught_exceptions()` is runtime evaluated • no change in user code to support `exception_ptr`'s functionality (only add constexpr)	• `constexpr int n = std::uncaught_exceptions()` can be confusing for some

P3820: split functions to runtime and consteval variant

positives	negatives
• no breakage in existing code • `const int n = std::uncaught_exceptions()` is runtime evaluated • `constexpr int n = std::uncaught_exceptions()` fails to compile	• user code needs to accomodate duality of functions and use `if consteval` to select right function • `const int n = std::consteval_uncaught_exceptions()` is still problematic • `constexpr int n = std::consteval_uncaught_exceptions()` still can confuse some

P3820: "constant when there is currently handled exception"

positives	negatives
• `const bool e = std::current_exception() != nullptr` is runtime evaluated • `const bool e = [] { try { ... } catch (...) { return std::current_exception() !=`	• **changes semantic of functions** and limits usability of using these to detect non-exceptional state • `std::current_exception() != nullptr` as detection if code is run inside exception handler will fail to **constant evaluate** outside exception handler, without workaround.

`nullptr; }}` is constant evaluated • `constexpr int n = std::uncaught_exceptions()` won't compile so it won't confuse anyone	• `const int n = ...` complex code with internal `std::uncaught_exception` can still be constant evaluated and it won't count runtime exception, which is **still a breaking change**

Ville's nuke constexpr from `uncaught_exceptions` and `current_exception` and add it later (in some form)

positives	negatives
• `const int n = std::uncaught_exceptions()` is runtime evaluated • `constexpr int n = std::uncaught_exceptions()` fails to compile, no one is getting confused	• limiting to basic functionality of throwing / catching / rethrowing • `std::uncaught_exceptions()` and `std::current_exception()` doesn't work in constant evaluated code at all • inability to temporary store exception and decide if it should be rethrow later, thus forcing existing code which could be easily upgraded to `constexpr` to invent workarounds to missing functionality • unless we change language rules significantly for `constexpr` local variables **the confusion will still be there, just later**

Implementation experience

The proposed solution was implemented in my clang prototype of `constexpr` exception for `std::uncaught_exceptions()`, and you can experiment with it at the compiler explorer.

The change itself was add to clang know in its evaluation state the evaluation is potentially-constant. And then the builtins implementing the exception handling function to detect it and fail to evaluate in that case.

There was a minor problem around initialization of a local `constexpr` variables, which for some reason are not cached, and are reinitialized as part of evaluation with same evaluation state. This lead to a funny error when following code didn't compile (reported to me by Ville):

```
const int n = []{
  constexpr int x = std::uncaught_exceptions(); // initialized twice, once duri
ng parsing
                                                // and second time during evalu
ation of the lambda
  return x;
}();
```

Clang when it sees initialization of a local `constexpr` variable it evaluates its initialization again, even when the value could be cached. This second evaluation in my first version was in potentially-constant state, and then the exception support function didn't work silently. Fix was changing the state of evaluation for initializers of local `constexpr` variables, and restoring previous one when the initialization is finished.

Wording

Change is add a new *Constant when* to `std::uncaught_exceptions()` and `std::current_exception()`.

17.9 Exception handling [support.exception]

17.9.7 Exception propagation [propagation]

```
using exception_ptr = unspecified;
```

1 The type `exception_ptr` can be used to refer to an exception object.

2 `exception_ptr` meets the requirements of *Cpp17NullablePointer* (Table 36).

3 Two non-null values of type `exception_ptr` are equivalent and compare equal if and only if they refer to the same exception.

4 The default constructor of `exception_ptr` produces the null value of the type.

5 `exception_ptr` shall not be implicitly convertible to any arithmetic, enumeration, or pointer type.

7 For purposes of determining the presence of a data race, operations on `exception_ptr` objects shall access and modify only the `exception_ptr` objects themselves and not the exceptions they refer to. Use of `rethrow_exception` or `exception_ptr_cast` on `exception_ptr` objects that refer to the same exception object shall not introduce a data race.

[*Note 2*: If `rethrow_exception` rethrows the same exception object (rather than a copy), concurrent access to that rethrown exception object can introduce

a data race. Changes in the number of `exception_ptr` objects that refer to a particular exception do not introduce a data race. — *end note*]

8 All member functions are marked `constexpr`.

```
constexpr exception_ptr current_exception() noexcept;
```

? *Constant When*: not evaluated within potentially-constant [expr.const] initialization.

9 *Returns*: An `exception_ptr` object that refers to the currently handled exception or a copy of the currently handled exception, or a null `exception_ptr` object if no exception is being handled. The referenced object shall remain valid at least as long as there is an `exception_ptr` object that refers to it. If the function needs to allocate memory and the attempt fails, it returns an `exception_ptr` object that refers to an instance of `bad_alloc`. It is unspecified whether the return values of two successive calls to `current_exception` refer to the same exception object.

[*Note 3*: That is, it is unspecified whether `current_exception` creates a new copy each time it is called. — *end note*]

If the attempt to copy the current exception object throws an exception, the function returns an `exception_ptr` object that refers to the thrown exception or, if this is not possible, to an instance of `bad_exception`.

[*Note 4*: The copy constructor of the thrown exception can also fail, so the implementation can substitute a `bad_exception` object to avoid infinite recursion. — *end note*]

```
[[noreturn]] constexpr void rethrow_exception(exception_ptr p);
```

10 *Preconditions*: `p` is not a null pointer.

11 *Effects*: Let *u* be the exception object to which `p` refers, or a copy of that exception object. It is unspecified whether a copy is made, and memory for the copy is allocated in an unspecified way.

(11.1) — If allocating memory to form *u* fails, throws an instance of `bad_alloc`;

(11.2) — otherwise, if copying the exception to which `p` refers to form *u* throws an exception, throws that exception;

(11.3) — otherwise, throws *u*.

```
template<class E> constexpr exception_ptr make_exception_ptr(E e)
noexcept;
```

12 *Effects*: Creates an `exception_ptr` object that refers to a copy of `e`, as if:

```
try {
  throw e;
```

```
  } catch(...) {
    return current_exception();
  }
```

```
template<class E> constexpr const E* exception_ptr_cast(const excep-
tion_ptr& p) noexcept;
```

14 *Mandates*: `E` is a cv-unqualified complete object type. `E` is not an array type. `E` is not a pointer or pointer-to-member type.

[*Note 6*: When `E` is a pointer or pointer-to-member type, a handler of type `const E&` can match without binding to the exception object itself. — *end note*]

15 *Returns*: A pointer to the exception object referred to by `p`, if `p` is not null and a handler of type `const E&` would be a match ([except.handle]) for that exception object. Otherwise, `nullptr`.

17.9.6 `uncaught_exceptions` [uncaught.exceptions]

```
constexpr int uncaught_exceptions() noexcept;
```

? *Constant When*: not evaluated within potentially-constant [expr.const] initialization.

1 *Returns*: The number of uncaught exceptions ([except.throw]) in the current thread.

2 *Remarks*: When `uncaught_exceptions() > 0`, throwing an exception can result in a call of the function `std::terminate`.

Feature test macros

No feature test macro added as this is a bugfix. I'm happy to add if LEWG asks.